

Freenet

A Peer-to-Peer Platform for Real-Time Decentralized Applications
Idempotent Commutative Monoids and Adaptive Small-World Routing

Ian Clarke

ian@freenet.org

<https://freenet.org>

August 9, 2026

Abstract

Freenet is a peer-to-peer platform for building real-time decentralized applications without global consensus. Its core abstraction is a *contract*: a WebAssembly module that defines both the public state of an application and the merge semantics used to reconcile replicas. The platform does not impose a single consistency model. Instead, each contract exposes an idempotent commutative merge operation (equivalently, a join-semilattice) and an application-defined summary/delta protocol for efficient synchronization.

Each contract has a location on a one-dimensional ring derived from the hash of its code and parameters, and is replicated on peers near that location. Requests are routed using a per-neighbor, distance-indexed performance model rather than topological distance alone. For private state, the platform provides *delegates*: local WebAssembly agents that hold secrets on the user's device and interact with contracts and other software through attributed messages.

This architecture targets decentralized applications such as group chat, collaborative documents, social feeds, and reputation systems, where availability and application-defined convergence are preferable to global linearizability. The paper describes the architecture implemented in the freenet-core reference runtime and presents a 24-hour snapshot of routing behavior on the deployed network. It also identifies deployed, experimental, and unimplemented mechanisms.

1 The Problem

The modern web is built on infrastructure owned by a small number of companies. Group chat, social feeds, collaborative documents, identity, and reputation all run on it. So does the power to moderate, surveil, and remove the people who use it. This paper describes a peer-to-peer platform aimed at providing the same kinds of services without the same single points of control.

Two broad families of decentralized systems have been deployed at meaningful scale, and neither is a good substrate for interactive applications.

Content-addressed storage. The original Freenet (Clarke et al., 2001) and IPFS (Benet, 2014) provide name-by-hash access to immutable data. Applications must add their own mechanisms for mutable state. Chat rooms, document editors, and social feeds consequently implement consistency independently, often with different behavior across clients.

Blockchains. Ethereum (Wood, 2014) and successor smart-contract platforms provide programmable mutable state, but their consistency model is global linearizable consensus. Every state transition is ordered with every other transition, and replicated by every participating node. The latency, throughput, and replication cost that follow are reasonable for value-transfer and authoritative ledgers, but they are too expensive for the interactive applications that constitute the bulk of the web.

The gap between these is large. A chat room does not need a global linear order on every message; it needs replicas that converge to the same content. A reputation system does not need every reader to download every signed record; it needs efficient summary-driven sync. A collaborative document does not need consensus on each keystroke; it needs an algebra of merge that the application defines.

This paper describes a peer-to-peer platform built around that observation. Applications carry their own merge semantics, expressed as sandboxed WebAssembly. The platform handles routing, replication, synchronization, transport, and sandboxing generically over whatever algebra the application supplies, alongside a separate primitive (the delegate) for private state and sensitive operations that should not be replicated. The result targets a class of decentralized applications that has historically been awkward to build on either content-addressed or blockchain substrates.

2 Design Thesis

The design separates state reconciliation from state propagation. Each application packages an idempotent, commutative merge operation on byte-string state as a WebAssembly module. The platform determines where that state lives, how peers locate it, how replicas exchange differences, how updates reach subscribers, and how secrets remain local. Its routing and synchronization mechanisms operate independently of the application-supplied state algebra.

This separation distinguishes the system from two common approaches to decentralized state.

Application-defined rather than fixed CRDTs. Conventional Conflict-free Replicated Data Type (Shapiro et al., 2011) frameworks provide implementations of counters, sets, registers, sequences, and other predefined lattices. A Freenet contract supplies its own merge function, validity invariants, and tiebreak rules. This permits workload-specific state representations, but the runtime cannot statically verify their algebraic properties: a contract with a non-associative merge function will not converge. Because contracts are content-addressed, such a failure remains attributable to a specific module that users can reject.

Per-contract consistency instead of global consensus. Blockchain platforms globally order state transitions. Freenet provides eventual consistency independently for each contract: replicas converge without coordinating the order of their updates with the rest of the network. This supports interactive applications whose updates can be merged, but not applications that require a single global order.

Separate public and private state. A chat room’s contents may be public while each user’s signing key remains private. Contracts hold replicated public state; delegates hold device-local private state. Messages between them carry platform-verified sender identities across an enforced trust boundary.

The remainder of the paper describes the primitives that follow from this thesis (Section 3), how updates move through the network (Section 4), how data is located (Section 5), how trust

is established and what it does not cover (Section 6), the current implementation status and open problems (Section 7), and the position in related work (Section 8).

3 Core Primitives

This section names the artifacts that appear in the rest of the paper and fixes notation.

3.1 Peers and the Ring

A *peer* is an instance of the Freenet Core running on some host. Each peer p has a *location* $\ell(p) \in [0, 1)$. Locations live on a one-dimensional ring with wrap-around distance

$$d(x, y) = \min\{|x - y|, 1 - |x - y|\}.$$

A peer's location is derived deterministically from its external network address; the exact derivation and its security implications are discussed in Section 6. Each peer maintains a set of *neighbor* connections, bounded between configurable minimum and maximum sizes (typically 25 and 200). Neighbor relations are reorganized over time by the topology rule described in Section 5.

3.2 Contracts

A *contract* $\mathcal{C}$ is a pair (code, params), where code is a WebAssembly module conforming to the platform's contract interface and params is an opaque byte string fixed at instantiation. The contract's *key* is

$$k(\mathcal{C}) = H(H(\text{code}) \parallel \text{params})$$

for a cryptographic hash H . The two-stage form lets a client that already holds $H(\text{code})$ derive the key for a new contract from new parameters without re-hashing the full WebAssembly module, which may be several megabytes. The contract's *location* $\ell(\mathcal{C}) \in [0, 1)$ is derived from $k(\mathcal{C})$ in the same way as a peer location. Each contract has a *state* $\sigma \in \mathcal{S}_{\mathcal{C}}$, replicated across peers near $\ell(\mathcal{C})$ on the ring. The contract's code defines $\mathcal{S}_{\mathcal{C}}$, the validity predicate $\text{valid}_{\mathcal{C}}$, the merge operation $\oplus_{\mathcal{C}}$, and the summary/delta synchronization functions (all detailed in Section 4). Contracts are public: their state is readable by any peer that can route to $\ell(\mathcal{C})$.

3.3 Delegates

A *delegate* is a WebAssembly module that runs on a single user's device, inside the local Freenet Core. It holds private *secret state* (sandboxed by the core) and responds to messages from user interfaces, contracts, and other delegates. The core attaches a verified sender identifier to every inbound message, so a delegate can apply policy based on sender identity in addition to message content. Delegates are local; they do not appear on the ring. Their trust boundary and cross-device replication mechanism are described in Section 6.

3.4 User Interfaces

A *user interface* is a web frontend (HTML, JavaScript, WebAssembly) delivered by a contract whose state encodes the application bundle. User interfaces run in the browser and communicate with the local core over a WebSocket API. They reach contracts and delegates only through that local core. They are not a distinct kind of network-visible artifact: from the platform's perspective, the bundle is just contract state with a particular MIME type.

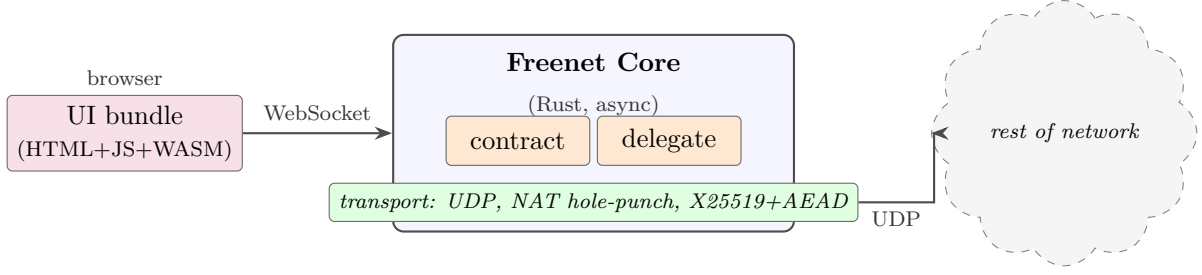


Figure 1: What runs in and around a single peer. The Freenet Core is a Rust process that hosts WebAssembly *contracts* (whose state is public and replicated across peers near the contract’s ring location) and *delegates* (whose state is local to the device and never leaves it). A *user interface* is an HTML/JS/WASM bundle delivered as contract state and run in the browser; it talks to its local Core over a WebSocket. The Core’s transport speaks UDP with NAT hole-punching to neighbors on the rest of the network.

3.5 Operations

A small set of operations connect these artifacts. We describe them informally here; their detailed lifecycles appear in Section 5.

PUT

Publish a contract: insert or update its state at the peers near $\ell(\mathcal{C})$.

GET

Retrieve a contract’s current state by key.

UPDATE

Submit an update to a contract. The update is merged into existing replicas using the contract’s merge function and propagated through the mesh of neighboring co-hosts.

SUBSCRIBE

Register interest in future state changes to a contract.

CONNECT

Establish a peering relationship with the network as a new node.

PUT, GET, UPDATE, and SUBSCRIBE all route over the ring; their destination is $\ell(\mathcal{C})$. CONNECT is the bootstrap operation by which a fresh peer joins the topology.

4 How Updates Move

This section describes the algebra each contract carries, the synchronization protocol that propagates it efficiently across replicas, the co-host mesh through which updates flow, and the leases that keep hosting aligned with demand.

4.1 Contract State as an Idempotent Commutative Monoid

Definition 1 (Contract state algebra). *A contract $\mathcal{C}$ defines:*

1. a state space $\mathcal{S}_{\mathcal{C}}$ (an abstract set; the wire encoding is a byte string and the contract code defines the encoding);
2. a validity predicate $\text{valid}_{\mathcal{C}} : \mathcal{S}_{\mathcal{C}} \rightarrow \{\perp, \top\}$;

3. an identity element $e_C \in \mathcal{S}_C$ with $\text{valid}_C(e_C) = \top$;
4. a binary merge operation $\oplus_C : \mathcal{S}_C \times \mathcal{S}_C \rightarrow \mathcal{S}_C$, which is associative, commutative, and idempotent, with e_C as identity.

An idempotent commutative monoid is equivalently a join-semilattice; the partial order is $a \sqsubseteq b \iff a \oplus_C b = b$. This is the algebraic structure of a state-based CRDT (Shapiro et al., 2011). The monoid presentation emphasizes the relevant operational property: peers can merge state at any time, in any order, with any multiplicity, and obtain the same result.

The validity predicate is checked at the boundary. A peer that receives a candidate state σ with $\text{valid}_C(\sigma) = \perp$ rejects it and never merges it into its local replica. Validity typically encodes integrity invariants beyond what the type system enforces, for example “every record in the log carries a valid signature by an authorized key.”

An *update* is itself an element of $\mathcal{S}_C$. When a peer holding state σ receives an update u , it computes $\sigma' = \sigma \oplus_C u$, subject to validity. There is no separate notion of operation: the algebra of state is the algebra of change.

Remark (Convergence). *For a finite multiset of valid updates $u_1, \dots, u_n \in \mathcal{S}_C$ and a starting state σ_0 , the merged result $\sigma_0 \oplus_C u_{\pi(1)} \oplus_C \dots \oplus_C u_{\pi(n)}$ is independent of the permutation π by associativity and commutativity, and idempotence makes duplicate delivery harmless. These are the sufficient conditions for the Strong Eventual Consistency theorem of Shapiro et al. (Shapiro et al., 2011): peers that have delivered the same set of valid updates converge regardless of order or repetition. Freenet applies this result to monoids supplied by individual contracts.*

4.2 Worked Examples

Monotone counter. $\mathcal{S} = \mathbb{N}$, $\oplus = \max$, $e = 0$. The partial order is the usual $\leq$.

Last-writer-wins register. $\mathcal{S} = V \times \mathbb{N} \times H$, where V is the value type, $\mathbb{N}$ a Lamport-style timestamp, and H a tiebreaker hash. Merge is lexicographic max on the (timestamp, hash) pair.

Signed log. $\mathcal{S}$ is a finite set of records, each record carrying a signature by an authorized key. $\oplus = \cup$, $e = \emptyset$. Validity rejects any state containing a record whose signature does not verify against a key listed in params.

Observed-remove set. $\mathcal{S} = (E, T)$ where E is a set of (elem, id) pairs and T a set of ids that have been tombstoned. $\oplus$ is pairwise union; the materialized set is $\{\text{elem} : (\text{elem}, \text{id}) \in E, \text{id} \notin T\}$.

A bounded signed log (for example, one that keeps the most recent N records) requires an eviction function determined by the merged set. If eviction instead depends on arrival order, the merge ceases to be idempotent. Contract authors are responsible for maintaining this property.

4.3 Where This Sits in the Design Space

Bayou (Terry et al., 1995) introduced application-supplied merge procedures in 1995, and CouchDB, Riak, and Git’s merge drivers also expose conflict resolution to application code. Freenet combines this approach with constraints required by an untrusted peer-to-peer environment.

Contracts bind merge code to the data identity through content addressing, execute it in a sandbox with fuel and memory bounds, and require it to satisfy the idempotent-commutative-monoid laws. Bayou permits more general merge procedures, while Freenet uses these restrictions to support convergence on untrusted peers.

Fixed-lattice CRDT frameworks such as Automerger (Automerger contributors, 2026), Y.js (Y.js contributors, 2026), and Antidote (Akkoorath et al., 2016) compose applications from a predefined catalog. Freenet allows a chat room, reputation system, or public file feed to define a lattice specific to its consistency requirements. The runtime consequently cannot verify the algebraic properties statically. A violation appears as failure to converge rather than as a structural error at deployment time.

Freenet therefore permits more flexibility than fixed-lattice frameworks and less than Bayou. The summary/delta interface developed next also lets contracts specialize their synchronization protocols.

4.4 Summary/Delta Synchronization

Pairwise merge is sufficient for convergence but, on its own, would require peers to ship full state when reconciling. For non-trivial contracts this is prohibitive. The platform therefore extends the contract interface with three additional functions that allow peers to exchange only the difference between their states.

Definition 2 (Sync interface). *In addition to the algebra of Definition 1, a contract provides*

$$\begin{aligned} \text{summarize}_C &: \mathcal{S}_C \rightarrow \Sigma_C, \\ \text{getDelta}_C &: \mathcal{S}_C \times \Sigma_C \rightarrow \Delta_C, \\ \text{applyDelta}_C &: \mathcal{S}_C \times \Delta_C \rightarrow \mathcal{S}_C, \end{aligned}$$

where Σ_C and Δ_C are contract-defined byte-string types.

The summary type Σ_C is intended to be much smaller than full state. For a signed log it might be the set of record identifiers, or a Merkle root over them; for a counter it might be the counter’s value; for a large observed-remove set it might be a Bloom filter, or a pair of Bloom filters over E and T .

For the two-step protocol below to be useful, the contract author is required to ensure that getDelta produces something that, when applied, dominates the other peer’s state in the partial order:

Property 1 (Sync soundness). *For all $\sigma_A, \sigma_B \in \mathcal{S}_C$, let $s = \text{summarize}_C(\sigma_B)$ and $\delta = \text{getDelta}_C(\sigma_A, s)$. Then*

$$\text{applyDelta}_C(\sigma_B, \delta) \oplus_C \sigma_A = \text{applyDelta}_C(\sigma_B, \delta).$$

Equivalently, applying δ at σ_B brings σ_B at least as far up the lattice as σ_A would have if merged in full. The platform does not verify this property; the contract is responsible for it.

4.5 The Two-Step Sync Protocol

Two peers A and B reconcile their states for a contract C as follows:

1. A computes $s_A = \text{summarize}_C(\sigma_A)$ and sends s_A to B .

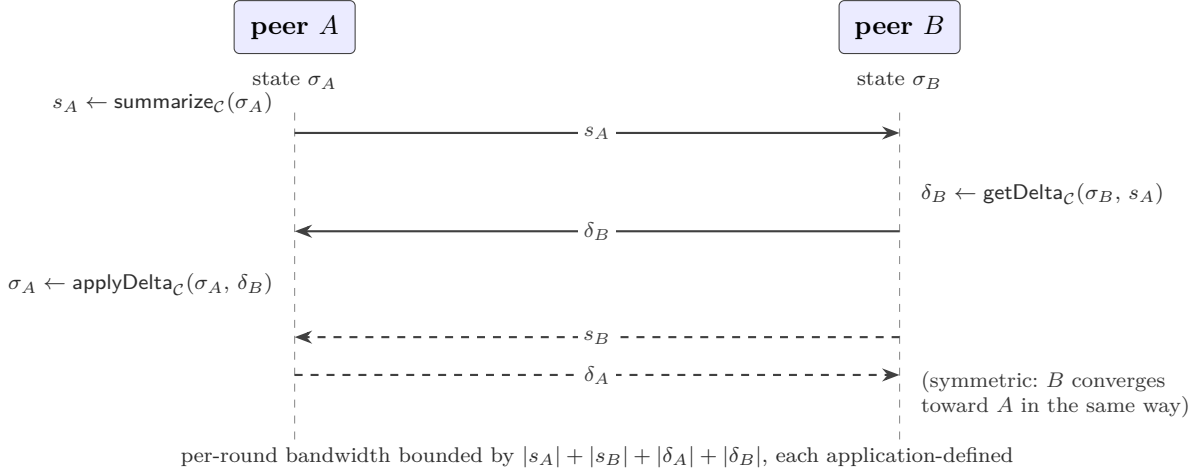


Figure 2: The two-step summary/delta sync protocol. Peer A sends a small summary s_A of its state. Peer B uses s_A to compute a *delta* relative to its own state and ships only the difference back, which A applies under the contract’s merge. The exchange is symmetric (dashed); each direction runs over the same transport. The contract supplies summarize_C , getDelta_C , and applyDelta_C , so the protocol is generic over whatever state representation the contract uses.

2. B computes $\delta_B = \text{getDelta}_C(\sigma_B, s_A)$ and sends δ_B to A . A applies the delta: $\sigma_A \leftarrow \text{applyDelta}_C(\sigma_A, \delta_B)$.

The protocol is symmetric: B runs the same exchange in the other direction. In practice the two directions are interleaved over a single connection. Bandwidth per reconciliation round is bounded by $|s_A| + |s_B| + |\delta_A| + |\delta_B|$, each of which is application-defined. Contracts that use approximate summaries may require multiple rounds before replicas fully converge (see the next subsection), in which case the total bandwidth is the sum of these per-round bounds.

4.6 Malformed Summaries and Malicious Deltas

The basic protocol reconciles honest replicas with stale state. Approximate summaries and malicious peers require additional consideration.

Approximate summaries. A summary type such as a Bloom filter has false positives: B may believe A already has a record that in fact A does not. In the bare protocol, the record is then not included in the delta and the two replicas remain divergent. Contracts that use approximate summaries must therefore either fall back to a more precise second round when divergence is suspected, or include the false-positive rate in their convergence guarantees and rely on subsequent reconciliation rounds (driven by later updates, subscription renewals, or explicit re-sync) to repair divergence. The platform offers no built-in fallback; the contract designs the round structure. Diagnosing a faulty or insufficient round structure is one of the open problems noted in Section 7.

Malicious peers. A peer that supplies a deliberately malformed summary, or returns a delta that violates the validity predicate, is treated like any other source of invalid state. The receiver applies valid_C to the merged result; if the result fails validity, the delta is rejected. A more subtle attack is for a peer to supply a delta that is valid but withholding (claiming to be up to date when it is not). The protocol cannot distinguish such a peer from one that has simply not

yet received the missing updates; the replication factor across multiple peers near $\ell(\mathcal{C})$ is the structural defense, not the sync protocol itself.

4.7 Subscriptions and the Co-Host Mesh

Once a contract is published, each hosting peer advertises that fact to its connected neighbors. An update arriving at any host is merged into local state and fanned out to neighbors that also advertise themselves as hosts of the contract, with each pairwise exchange using the summary/delta protocol above. These advertised co-host links form the update mesh. The live fan-out target set is derived from co-host advertisements rather than from a separate subscriber forwarding list; periodic reconciliation of the advertisements repairs a missed announcement or retraction.

Subscriptions serve a different purpose: they express demand and maintain soft-state paths from interested peers toward the contract’s home location. A peer on such a path hosts and advertises the contract, so the path becomes part of the same co-host mesh as every other replica. Hosting follows demand and eviction, while subscriptions are explicit and lease-bound.

Because $\oplus_{\mathcal{C}}$ is commutative, simultaneous updates injected at multiple points of the mesh do not require coordination. They propagate across co-host links and merge; the order in which different updates arrive at a given node does not affect its final state. In the deployed configuration a subscription lease lasts 8 minutes and a background task renews it every 2 minutes — but only while the contract is still *in use* at that peer, meaning it has a local client subscription, a registered downstream subscriber, or recent genuine GET or PUT access. A hosting peer with no remaining interest stops renewing; its lease lapses and the demand path collapses inward toward $\ell(\mathcal{C})$. A subscriber whose lease expires must re-subscribe. This interest-gating keeps the live subscription set proportional to active demand rather than to accumulated cache: orphaned paths die out within a small multiple of the lease period.

4.8 Relationship to Other Mechanisms

Versus operation-based CRDTs. An operation-based CRDT propagates individual operations and relies on a causal-broadcast layer to ensure exactly-once, causally ordered delivery. The protocol here propagates state and tolerates arbitrary loss, reordering, and duplication; the merge function does the work that causal broadcast would otherwise have to.

Versus Merkle anti-entropy. Distributed key-value stores commonly use Merkle-tree anti-entropy (DeCandia et al., 2007) to discover differences between replicas. That is a special case of the protocol here in which the summary is a hash tree and the delta is the subtree the other replica is missing. A contract whose data structure is amenable to a more compact summary (a list of identifiers, a counter value) pays a single round trip rather than $O(\log n)$ of them.

Versus rsync. `rsync` (Tridgell, 1999) reconciles files by exchanging block-level checksums. It is generic but oblivious to application structure. Summary/delta is the same shape of protocol lifted to a level where the application defines the granularity.

Versus delta-state CRDTs. The closest formal analogue is the delta-state CRDT literature (Almeida et al., 2018), which ships only changes between replicas of a fixed-lattice CRDT. The differences here are that the lattice and the delta-encoding are both supplied by the contract, and the result is embedded in a small-world routing layer rather than a fully-connected replica set.

5 How Data Is Found

A request for contract $\mathcal{C}$ originates at some peer and must reach a peer that hosts $\mathcal{C}$'s state. This section describes the ring topology, the rule that constructs it, the routing logic that uses it, and the bootstrap by which a new peer joins. The section closes with a 24-hour measurement of routing path lengths on the live network.

5.1 Locations and the Ring

A peer's location $\ell(p)$ is computed by hashing its external network address into $[0, 1)$. A contract's location $\ell(\mathcal{C})$ is similarly derived from its content-addressed key. Distance is the wrap-around metric of Section 3.

This arrangement makes locations stable across restarts that preserve a peer's address. Locations change when a peer is rehomed, when NAT mappings shift, or when a peer is reachable on multiple addresses. An adversary controlling many addresses (a cloud subnet, an IPv6 range, or a small botnet) can grind locations and concentrate them in chosen regions of the ring. Sybil resistance is left to application-level mechanisms such as signed contributions, reputation contracts, and capacity-limited delegation (Section 6).

5.2 Why Small-World Routing

The routing strategy derives from Kleinberg's small-world result (Kleinberg, 2000). Consider N peers on a ring (or any metric space), each with a constant number of *short-range* links to immediate neighbors and a small number of *long-range* links sampled with probability proportional to $1/d$ in ring distance d . Kleinberg proved that a node with knowledge limited to its neighbors can greedily route to any target in $O(\log^2 N)$ hops. The result requires neither global topology knowledge nor flooding, provided the link distribution remains approximately $1/d$.

The $1/d$ distribution is what makes the bound work. Too short and the long-range links don't help cover ground; too long and they overshoot the target. $1/d$ produces, at each scale, a constant probability of a useful long-range hop, so a request roughly halves its remaining distance at every step until the local neighborhood dominates.

Kleinberg's analysis assumes the $1/d$ distribution. Symphony (Manku et al., 2003) sampled long-range links from a harmonic distribution at join time and retained them. Freenet instead adjusts each peer's neighborhood continuously. A peer directs new **CONNECT** requests toward under-represented regions of its current connection distribution (*gap-targeting*) and prefers incoming candidates that close large gaps (*Kleinberg-style scoring*). These mechanisms maintain the distribution under churn.

5.3 Topology Construction

The **CONNECT** operation constructs the small-world topology. A joiner chooses a *desired location*, routes a **CONNECT** message greedily toward it, and receives acceptance decisions along the route and at its terminus. Target selection and acceptance both shape the topology toward Kleinberg's $1/d$ distribution (Kleinberg, 2000).

Desired location. A peer with fewer than K_0 existing connections targets its own location, with random jitter applied after consecutive acceptance failures so that subsequent attempts probe different ring regions. Joiner locations are derived from network-address prefixes (Section 6),

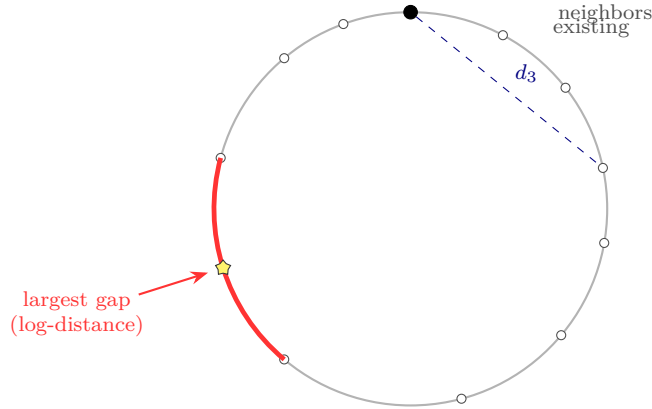


Figure 3: Gap-targeting from a single peer’s view. The peer measures ring distance d_i to each existing neighbor (one such d_i shown dashed), identifies the largest gap in log-distance space (red arc), and chooses its midpoint as the next **CONNECT** destination (star). The peer thus pulls its own connection distribution toward $1/d$ as it accumulates neighbors.

so distinct joiners hit distinct early targets and bootstrap traffic naturally spreads across the gateway’s neighborhood. A peer with at least K_0 connections instead computes a *gap target*: it measures the ring distance from itself to each existing neighbor, identifies the largest gap in that distribution in log-distance space, and targets the gap’s midpoint. Gap-targeting is the active mechanism by which each peer pulls its own connection distribution toward $1/d$ as it accumulates neighbors: the locations under-represented by current connections are exactly the locations the next **CONNECT** targets. The deployed thresholds are $K_0 = 3$ for the initial-join path and $K_0 = 5$ for the steady-state maintenance loop; the slightly higher maintenance threshold lets the local connection distribution settle before refresh **CONNECT**s start gap-targeting.

Acceptance. A peer that receives a **CONNECT** applies a chain of decisions. If it can forward strictly closer to the target, it forwards; this is the original terminus rule. A peer within a small fraction of the ring of the target (the deployed band is 5%) may additionally accept with a proximity-scaled probability while still forwarding, giving the joiner more than one nearby connection per request and softening the strict-terminus discipline. When a request reaches its terminus, the receiver’s choice depends on its current connection count: well below a small threshold it accepts unconditionally to unblock bootstrap; below the configured minimum-connections target it filters by a Kleinberg-style score on the candidate’s location relative to its existing neighbors, with the acceptance floor sliding from generous (early) to selective (near full); at or above the minimum it admits only the best-scoring candidate from a rolling window of recent arrivals. Above the maximum it rejects. When the terminal receiver rejects, the **CONNECT** may also be re-forwarded *uphill* (away from the target, up to a small budget of additional hops, deployed value 8) so that a joiner whose precise target lies behind a saturated peer can still be picked up by a nearby non-saturated one.

Formation of long-range edges. Gap-targeting and Kleinberg-score acceptance jointly produce the long-range component of the connection distribution. A peer with mostly local neighbors has its largest log-distance gap at long range, so its next **CONNECT** targets a distant terminus. The receiver favors candidates whose locations close large gaps in its own distribution.

Nearest-neighbor lattice. Kleinberg’s bound also assumes a short-range base lattice. Each production peer therefore seeks and retains its nearest connected successor and predecessor

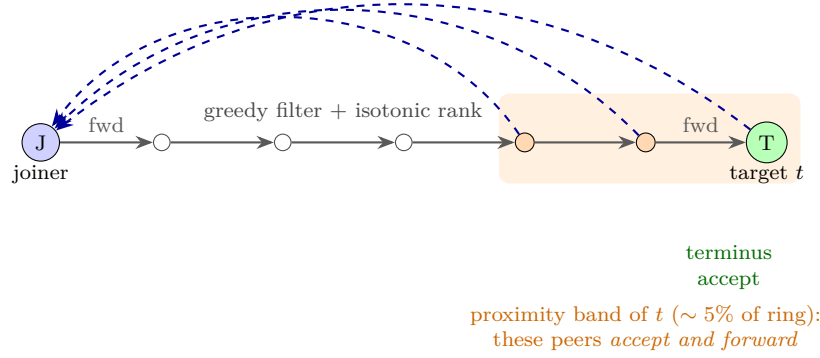


Figure 4: A single **CONNECT** from joiner J to its desired location t . Each forwarding hop applies a hard greedy filter (candidates strictly farther from t than the current peer are dropped) and then ranks the survivors with an isotonic-regression estimator trained on past forwarding outcomes; the request is sent to the best-scoring eligible neighbor (solid arrows). Once the route enters the proximity band of t (the deployed band is $\sim 5\%$ of the ring, shown shaded), each receiver may *additionally* accept the joiner with a proximity-scaled probability while still forwarding the request onward (dashed return edges). The terminus applies its acceptance chain (§5); a rejected terminus may re-forward the request uphill for a small budget of hops. Net result: one **CONNECT** request can establish multiple connections close to t , not just one at the terminus.

on the ring. Maintenance **CONNECT**s routed toward the peer’s own location discover closer candidates on either side; a strictly closer candidate can be admitted even at capacity, and ordinary score-based pruning does not discard the closest live edge on either side. The other connections remain available for the long-range distribution described above. The lattice is active when the configured connection budget is at least the deployed minimum of 25; sparse simulations with smaller budgets leave it disabled so that two reserved short links do not consume most of the topology.

Bounded fan-out. Admission control normally bounds the connection count between configurable minimum and maximum sizes (deployed defaults 25 and 200). Tightening both lattice sides may transiently exceed the maximum by at most two connections; maintenance then removes non-lattice links to return to the configured bound.

5.4 Greedy Forwarding

A request from peer p to target location t proceeds as follows. Let $N(p)$ be p ’s current set of neighbors. If some neighbor $q \in N(p)$ is strictly closer to t on the ring than p itself, forward the request to the closest such q ; otherwise p is a terminus for the request and serves it locally.

Each request carries a *hops-to-live* (HTL) counter, deployed maximum 10, that decrements on each forward and bounds how far a request travels: when HTL reaches zero the current peer stops forwarding and answers from local state or reports a miss. HTL is a termination bound only. Every forward is greedy — performance-ranked once the adaptive layer of the next subsection has enough history — and a peer that is near t but has not cached the contract forwards to the next-best candidate under a skip-list that prevents revisiting, rather than switching to a random walk. (Topology-mixing randomness lives in the **CONNECT** path of Section 5, not in request forwarding.)

5.5 Adaptive Routing

Ring distance alone does not identify the fastest path because neighbors may differ by an order of magnitude or more in bandwidth, RTT, and uptime. The router therefore selects the next hop by predicted total cost.

The local peer maintains a rolling history of routing events of the form (q, d, τ, b, f) , where q is the selected neighbor, d is its ring distance to the request’s target, τ is the observed time-to-first-byte, b is the observed transfer rate, and f is a failure indicator. From this history the router fits three distance-indexed baseline estimators:

$$\hat{\tau}_q(d), \quad \hat{p}_q(d), \quad \hat{b}_q(d),$$

the predicted latency, failure probability, and throughput as functions of distance-to-target. Each is fit by *isotonic regression* (Barlow et al., 1972; de Leeuw et al., 2009), the standard non-parametric estimator for a monotone function.

The choice of isotonic regression encodes a structural prior we are willing to commit to (latency and failure are non-decreasing in distance-to-target, on average) without committing to a parametric family for the shape of the curve. The pool-adjacent-violators algorithm fits each curve in time logarithmic in history size; old events age out so the model tracks current conditions. Per-peer adjustments account for stable differences among neighbors that the global distance curve does not capture.

The router blends these isotonic baselines with predictions from Renegade (Clarke, 2026), a k-nearest-neighbor model whose features identify the peer, contract location, distance, and observation time. Renegade predicts the same three outcomes rather than a residual correction. For each outcome y the combined prediction is

$$\hat{y} = (1 - w)\hat{y}_{\text{iso}} + w\hat{y}_{\text{knn}}, \quad 0 \leq w \leq \frac{1}{2},$$

where w grows as that prediction stage accumulates observations. The isotonic component supplies the monotone distance prior and the k-nearest-neighbor component captures interactions that distance alone cannot express, such as a peer that performs normally for most contracts but selectively drops requests for one region of the key space.

Because the models consume end-to-end routing outcomes, their predictions combine the neighbor’s link quality (RTT, bandwidth, and reliability) with the quality of onward paths through that neighbor. Onward-path quality changes more rapidly under churn because it depends on peers beyond the immediate link. The rolling history removes stale observations as new events re-score the neighbor.

Given a target t for a new request, the router selects the neighbor that minimizes an expected-cost score combining the three blended predictions. A neighbor with too little history falls back to greedy-by-distance until enough events have accumulated. The isotonic predictions use linear interpolation between fitted points and extrapolate linearly beyond the training-data domain.

Scope of adaptive routing. Once sufficient history is available, ordinary requests (GET, PUT, SUBSCRIBE, and UPDATE) select neighbors by predicted cost. CONNECT retains a hard greedy filter: only neighbors strictly closer to the target than the current peer are eligible. A separate isotonic estimator trained on accept/reject outcomes ranks those candidates. The filter keeps topology-construction traffic moving toward its target; ordinary requests can use the broader candidate set.

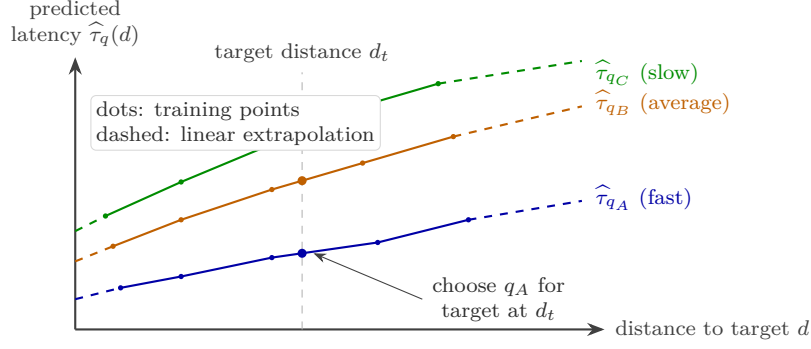


Figure 5: The isotonic baseline used by adaptive routing. Each neighbor q has a non-parametric, non-decreasing latency estimate $\hat{\tau}_q(d)$ as a function of ring distance to the target, fit by isotonic regression (pool-adjacent-violators) on the history of routing events. The fitted points (dots) are connected by linear interpolation; beyond the training-data domain the model extrapolates linearly (dashed). Two more isotonic baselines track predicted failure probability and throughput; one outcome is shown for clarity. For a request to a target at distance d_t , the router reads each neighbor’s prediction at d_t and picks the neighbor that minimizes the combined expected cost after blending these baselines with the k-nearest-neighbor predictions described in the text. The chosen neighbor need not be the one topologically closest to the target.

5.6 Lifecycles

The five operations of Section 3 are realized as transaction state machines coordinated by the local core. Each is assigned a transaction id, tracked through completion, and supports timeout, retry, and composite parent/child relationships.

CONNECT. As above. A successful **CONNECT** establishes a single neighbor relationship. A joining peer issues an initial sequence of **CONNECT** operations to populate its first K_0 neighbors using own-location targeting; once it has enough connections for gap analysis, all subsequent **CONNECT** operations use gap-target locations. Topology growth and refresh are driven by additional **CONNECT** operations issued from a maintenance loop, not as a side effect of ordinary request traffic.

PUT. A **PUT** carries a contract $\mathcal{C}$ and an initial state σ_0 . The originator validates and stores the state, computes its contract-defined summary, and routes a probe toward $\ell(\mathcal{C})$. An intermediate peer that does not already hold the contract forwards the probe without storing the state. If the probe finds a fresh holder, that peer returns its summary; the originator computes the corresponding delta and sends it to the holder, which merges it through the ordinary **UPDATE** path and co-host fan-out.

If the probe finds no fresh holder, fails, or cannot use the summary-first protocol with the next peer, the operation falls back to the full-state **PUT** path. On that path, routed peers validate and host the state as before. Both paths remain subject to the hosting budget, and there is no separate cache tier beneath hosting. A peer that begins hosting announces the fact to its neighbors (Section 4).

GET. A **GET** carries a key k . The originating peer routes toward the corresponding location. Any peer along the path that currently hosts the contract may answer with its current state σ and parameters; the response travels back along the request path, optionally caching at intermediate peers.

A route that terminates without finding a host does not immediately report a miss. The terminus consults its neighbors’ host advertisements (Section 4) and forwards to a neighbor that holds the contract, if one exists. Greedy forwarding reaches the relevant ring neighborhood; the advertisement mesh supplies the final step when the selected peer is not a host. **SUBSCRIBE** terminates the same way.

The requester can verify on arrival that the served code and parameters hash to k . *State* is not directly verifiable from the key: the key is derived from code and params, not from state, and state is mutable. This is a meaningful difference from immutable content-addressed stores such as IPFS, where `get( $k$ )` returns a value whose own hash equals k . Readers carrying that mental model should note that the guarantee does not transfer to Freenet contracts. State is trusted only to the extent that the contract’s validity predicate accepts it; replication across several peers near $\ell(\mathcal{C})$ is the structural defense against a single peer serving a stale or doctored state.

UPDATE. An **UPDATE** carries a key k and a candidate state $u \in \mathcal{S}_{\mathcal{C}}$. It is routed toward $\ell(\mathcal{C})$. Each peer along the path that already hosts the contract merges u into its local copy subject to the validity check, then propagates the change to neighboring co-hosts (Section 4).

SUBSCRIBE. A **SUBSCRIBE** establishes a lease on future updates. The subscriber routes the request toward $\ell(\mathcal{C})$ and the first hosting peer responds; intermediate peers along the path record the subscription. Subscriptions are leased and renewed periodically (Section 4).

5.7 A 24-Hour Snapshot of Routing on the Deployed Network

This subsection records a single observation of routing behavior on the deployed network. It is a sanity check, not a scaling study. We also include a snapshot of the connection-distance distribution (Figure 6), a direct empirical check on the small-world topology described above. The 24-hour window ends 2026-05-24. Network size in that window was 443 distinct active peers in the last 10 minutes, with 604 distinct peer identifiers seen across the full 24 hours (the difference attributable to churn). All peers run with telemetry enabled in the current deployment.

For each **GET** and **PUT** transaction we compute a *path length*: the number of distinct peers seen emitting the corresponding `*_request` event for that transaction id, which is the number of peers visited (hops + 1). Statistics over operations whose telemetry reached completion in the window:¹

Operation	n	mean	median	p95	p99
GET	1,302	6.84	7	11*	11*
PUT	158	6.01	7	10*	10*

The deployed hops-to-live cap is 10, so any route that would otherwise visit 12 or more peers is dropped before it can appear in the table. The reported p95 and p99 for **GET are therefore at the cap itself; they record where the tail is truncated, not where it actually lives. We are not directly observing the upper tail of the path-length distribution.*

The measurement has two further limitations. First, it covers one observation at one network size. At $N \approx 443$, $\log_2 N \approx 8.8$; the observed median path length of 7 peers (6 hops) for

¹In this May 2026 snapshot, **UPDATE** (median path length 2, $n = 386$) and **SUBSCRIBE** (median 1, $n = 5,359$) propagated primarily through pre-existing subscription paths rather than fresh greedy routes, so their path lengths reflected the distance from the subscriber to the nearest hosting peer rather than routing-layer hop counts. They are excluded from the table here.

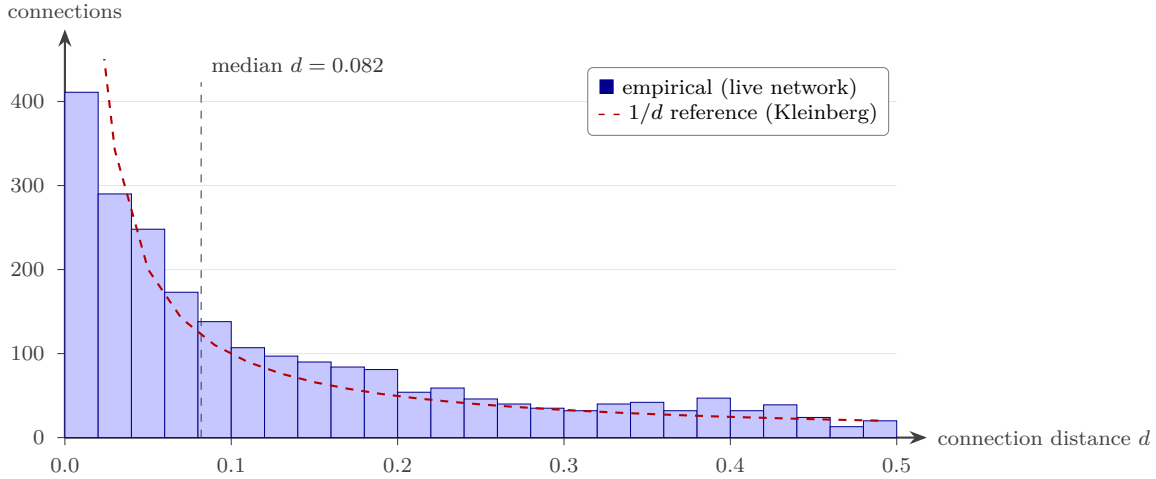


Figure 6: Distribution of per-connection ring distances on the deployed network. Each of the $n = 2274$ live connections across $N = 396$ active peers (snapshot taken 2026-05-25, source: the telemetry collector at `nova.locut.us:3133`) contributes one data point; bins are linear, width 0.02. The dashed curve is a Kleinberg $1/d$ reference normalized to the same total mass. The empirical distribution is monotonically decreasing in d , concentrated at short range with a long-range tail — the qualitative small-world shape that the active gap-targeting and Kleinberg-style acceptance of Section 5 are designed to produce. This is a consistency check, not a validation of optimality: Kleinberg’s bound is sensitive to the tail *exponent*, and greedy routing degrades when the tail is too heavy as much as when it is too light. The empirical curve lying *above* the $1/d$ reference in the tail therefore indicates more long-range mass than a pure $1/d$ law, which is not per se better; fitting the exponent and relating it to hop count belongs to the scaling study deferred in Section 5.7, not to this single snapshot. Median connection distance is 0.082; the maximum half-ring distance is 0.5.

GET is consistent with logarithmic-diameter routing, but a single data point at a single N cannot distinguish $O(\log n)$ from $O(\sqrt{n})$, $O(n^{0.4})$, or a constant. Second, the table does not compare against any baseline, so it does not isolate the contribution of adaptive routing from greedy-by-distance.

A controlled simulation sweep across orders of magnitude in N is needed to test the routing claims and compare greedy-by-distance with adaptive routing. The deterministic harness of Section 7 currently validates correctness and convergence at the tens-of-nodes scale, but it does not emit the required hop-count-versus- N curve. Extending it to do so is listed as an open problem in Section 7.

6 Security and Trust Boundaries

The platform has several layers, each with distinct authentication and trust assumptions. This section defines their guarantees, boundaries, and excluded threats.

6.1 Transport: Hop-by-Hop Encryption and Authentication

A connection between two neighboring peers is established by an authenticated ephemeral Diffie-Hellman handshake (X25519). The handshake exchanges symmetric keys used to encrypt subsequent traffic with AEAD constructions: AES-128-GCM by default, with ChaCha20-Poly1305 as an alternative for platforms without AES hardware acceleration. The handshake

itself is encrypted with the recipient’s long-term identity key, which doubles as a defense against unsolicited probes: a peer that has not received a properly-encrypted hello does not respond at all, making the port effectively invisible to a passive scanner.

This authentication is *hop-by-hop*. A request that traverses peers $A \rightarrow B \rightarrow C \rightarrow D$ is authenticated by A to B , by B to C , and by C to D . There is no transport-level guarantee that the message body at D is the body A originally sent: an intermediate peer can drop, reorder, or modify the payload before forwarding.

6.2 Application Layer: End-to-End Authentication via Validity Predicates

End-to-end integrity is the contract’s responsibility. A contract’s validity predicate valid_C runs at every peer that touches the state. Update messages typically include a signature by an application-authorized key over the update content; the validity predicate verifies the signature and rejects the update otherwise. Because valid_C is itself part of the contract’s content-addressed code, a peer cannot serve a state that fails validity without being caught by every receiving peer that runs the same code.

The combination is the platform’s substitute for a single end-to-end transport channel. Transport gives hop-by-hop confidentiality and authentication of neighbors. The contract gives end-to-end semantic integrity of state.

6.3 NAT Traversal and Bootstrap

Most peers operate behind consumer NAT and do not have a stable inbound address. The transport supports hole-punching: when two peers want to connect and at least one is behind a NAT, both peers send hello packets toward each other’s observed external addresses simultaneously (Ford et al., 2005). The introducer in this scheme, the peer that reports each side’s externally-observed address, is whichever peer routed the connection request between them.

For a brand-new peer joining the network the introducer is a gateway, which is required to have a public IP. For a peer that already has at least one connection, the introducer is one of its existing neighbors. Gateways are required only for the initial join. Existing peers introduce all subsequent connections, so the network can continue forming connections while every gateway is offline.

A peer’s location is derived from this externally-observed address. The implications are discussed in the next subsection.

6.4 Location Assignment: A Known Sybil and Grinding Surface

A peer’s location is computed by hashing its externally-observed network address into $[0, 1)$, but the hash is taken over the address *prefix* rather than the full address: the high 24 bits of an IPv4 address (so all hosts in the same /24 share a location) and the high 48 bits of an IPv6 address (the typical site-level allocation). The intent is to raise the cost of location grinding: an adversary that wants to place peers at chosen ring locations needs control of multiple distinct prefixes, not merely many hosts inside one subnet.

Prefix hashing only raises the cost of the attack. An adversary who can rent IP space across many /24s, obtain an IPv6 range, or provision VMs across distinct cloud prefixes can still grind locations and concentrate peers in chosen ring regions. Those peers can selectively answer contract requests or crowd honest peers out under the topology rule.

Prefix hashing also places every honest peer sharing a prefix at the *same* ring location. Independently operated peers behind a university or corporate NAT, a carrier-grade NAT

pool, or a popular cloud region consequently have ring distance zero from one another. This common deployment case affects topology construction, replication, and the application-layer Sybil mitigations described below.

Topology construction. Gap-targeting (Section 5) assumes that neighbor locations are distributed around the ring. A cluster of co-located peers appears as a zero-width spike to the largest-log-distance-gap computation. The nearest-neighbor lattice makes the closest successor and predecessor especially important: an adversary able to grind locations on both sides of a target can occupy those predictable short-range slots and attempt a local eclipse, although ordinary liveness eviction still removes failed links. *Replication.* Peers near a contract location are intended to protect against a single peer serving stale or doctored state. Co-located peers may instead share an operator and failure domain, reducing the independence of those replicas. *Sybil bootstrapping.* Application-layer defenses depend on honest peers being geographically diverse near critical locations, which shared prefixes make less likely.

Adding a per-peer nonce would spread co-located peers but would also restore the grinding freedom that prefix hashing is intended to reduce. Resolving this tension remains open (Section 7).

The platform does not solve this at the location layer. Two architectural mitigations operate at higher levels.

First, the contract layer’s validity predicates make state-doctoring detectable: a malicious peer that returns a state with an invalid signature is rejected by every honest receiver. The attack surface this leaves open is selectively serving stale (but still valid) state, not actively forging state.

Second, the platform expects identity, reputation, and contribution-control to be built as application-layer contracts (see Ghost Keys (The Freenet Project, 2026a) for an example of a contract-mediated anonymous identity scheme). A contract that posts authoritative information can require its writers to hold reputation tokens issued by another contract; an inbox contract can demand a small proof of work; a community forum can require introductions from existing members. The platform’s role is to provide a substrate on which these mechanisms can be built, not to embed any particular one in the routing layer.

The current design leaves Sybil defense to applications. A platform-level scheme would either impose a particular economic or identity model or provide only a generic obstacle that a sufficiently resourced adversary could overcome.

Application-layer contracts are themselves routed and replicated through the vulnerable location layer. An attacker with enough /24 prefixes to crowd a contract’s location can serve poisoned reads and prevent honest writes from reaching its canonical replicas. Validity predicates catch forgery but not selective service of stale, valid state. Bootstrapping therefore requires a property the platform does not currently provide: geographic diversity near safety-critical contracts, sufficient ring-distance separation among their replicas, or out-of-band trust anchors such as a bundled root key or release-managed seed list. Selecting and implementing such a mechanism remains open work.

6.5 Delegates: A Trust Boundary Around Secrets

Contracts hold public state. Applications also have private state (a user’s signing keys, a private contact list, the decryption material for an end-to-end encrypted inbox) that should never be replicated and should be accessible only through controlled interfaces. The platform’s primitive for this is the delegate.

A delegate is a WebAssembly module running inside the local Freenet Core on a user’s device. It holds private secret state, sandboxed by the core. Other components (user interfaces, contracts, other delegates) can interact with it only by sending messages. The core attaches a

verified sender identifier to each inbound message, so the delegate’s policy can be “only respond to messages from UI X, originating from contract Y.”

The trust boundary is therefore platform-enforced rather than conventional. Traditional in-process encapsulation (a “private” field on an object) is honored only by code that chooses to honor it; the same address space affords access to memory, file system, and undocumented APIs. A delegate’s secret state is not addressable from any other code on the device. The only way to extract a secret from a delegate is to convince the delegate, on the basis of sender identity and message contents, that releasing the secret is allowed.

The boundary is enforced by the local Freenet Core, which is itself a piece of software running on the same host. A compromised or tampered Core can violate the boundary directly. Defending against this requires hardware-attested execution or TEE-style mechanisms that the platform does not currently provide; the trust model here assumes an uncompromised local Core.

Worked example: signing without exporting a key. A chat application’s UI needs to send signed messages. Without delegates, it must hold the private key. With delegates, the UI sends a (sign, payload) message to a key-manager delegate; the delegate verifies the sender, signs the payload, and returns the signature. The private key never crosses the boundary. A bug or compromise in the UI can produce unwanted signed payloads but cannot exfiltrate the key itself; the delegate is also free to apply rate limits or user confirmation prompts before signing.

Cross-device replication. A user typically runs the “same” delegate on multiple devices, and the platform’s intended pattern for keeping those instances in sync is a shared-secret contract: a contract that holds an encrypted, replicated representation of the shared private state, decryptable only by the delegates that hold the symmetric key. The contract’s merge function provides convergence; the delegate provides confidentiality. This pattern is only partially implemented at the time of writing (see Section 7); the encrypted-contract primitives exist but the end-to-end delegate-to-delegate flow is not yet wired up.

6.6 Resource Accounting and What It Doesn’t Cover

Each peer enforces limits on bandwidth (token-bucket rate limiting at the transport layer), storage (a byte-budgeted set of hosted contracts), CPU, and memory (per-contract WebAssembly fuel and explicit memory bounds, enforced by Wasmtime (Bytecode Alliance, 2026)). The storage budget is denominated in hosted-state bytes and defaults to a clamped fraction of host memory because resident memory and update fan-out constrain the number of active contracts more than disk space does. An operator may override the default. When over budget, a peer removes the contract with the fewest dependent subscribers, breaking ties by least-recent genuine access. Excessive resource consumption by a neighbor results in disconnection.

These limits protect against a single misbehaving neighbor, but not coordinated traffic from many neighbors or denial of service through many cheap identities. Addressing those attacks probably requires accounting that persists across identities, as discussed in Section 7.

7 Implementation Status and Open Problems

This section describes the reference implementation, reports the status of each major mechanism, and lists unresolved architectural questions.

7.1 Implementation

The Freenet Core is implemented in Rust as an asynchronous `tokio`-based runtime. Contracts and delegates execute inside Wasmtime (Bytecode Alliance, 2026); both run under per-instance fuel limits and explicit memory bounds so that a runaway or malicious module cannot exhaust the host. Contract state is persisted through a pluggable store layer with implementations on top of `redb` (Berner, 2026) and `SQLite`.

The runtime is organized as a single central event loop, a `tokio::select!` over inbound transport packets, client API events, operation timers, and background tasks, that dispatches into per-operation state machines. The single-coordinator design simplifies reasoning about partial state but limits the throughput of a single peer to what one event-loop thread can sustain. So far this has not been the binding constraint in deployment.

Deterministic simulation testing. A peer-to-peer system is hard to test by ordinary means: many bugs in routing, congestion, or convergence appear only in specific multi-peer interleavings that are difficult to reproduce. The Core ships with a deterministic simulation harness built on Turmoil (Tokio Project, 2026): virtual time, a seeded global RNG, an in-memory socket implementation, and fault-injection knobs for loss, latency, partitions, and reorderings. Tests run the entire network in a single deterministic process and replay bit-for-bit from a seed. Most non-trivial routing, subscription, and contract-execution bugs we have found were first reproduced in simulation.

Content-addressed components. Contracts, delegates, and UI bundles are all addressed by hash. This pushes a lot of complexity off the platform: there is no version negotiation, no upgrade protocol, no need for the platform to understand application semantics during deployment. A new version is published under a new key; the old key continues to work as long as anyone hosts it.

7.2 Implementation Status of Each Mechanism

The following table reports the current status of the mechanisms described in this paper. “Deployed” means present in the production runtime and exercised on the live network; “Implemented” means present in the runtime but not yet broadly exercised at scale; “Experimental” means available as a build-time option but not in production; “Open” means designed or partially designed but not implemented.

Mechanism	Status	Notes
Small-world ring topology	Deployed	Nearest-neighbor lattice plus Kleinberg-style long links
Greedy forwarding, HTL-bounded	Deployed	HTL cap = 10; greedy/performance-ranked every hop
Two-stage CONNECT acceptance	Deployed	Greedy forward, accept-and-forward in 5% band, Kleinberg-scored at terminus
Adaptive ensemble routing	Deployed	Weighted blend of isotonic and Renegade k-nearest-neighbor predictions
Summary/delta synchronization	Deployed	Per-contract, application-defined
Summary-first PUT	Deployed	Probe and delta for existing state; full-state fallback
Demand paths with leases	Deployed	8-min lease, 2-min renewal, interest-gated
Neighbor co-host mesh	Deployed	Update fan-out and terminus lookup from host advertisements
Non-idempotency containment	Deployed	Bounded commit and egress suppression after detection
Contract WebAssembly execution	Deployed	Wasmtime, fuel + memory limits
Delegates (local, secret state)	Deployed	Used by River chat (The Freenet Project, 2026b)
Delegate cross-device sync via shared-secret contract	Partial	Encrypted-contract primitives exist; end-to-end delegate flow not wired up
NAT hole-punching transport	Deployed	UDP, X25519 + AEAD
Fixed-rate congestion control	Deployed	Production default
LEDBAT++ congestion control	Experimental	Did not meet our requirements in initial trials; not in production
BBR-style congestion control	Experimental	Build-time option, not the default
Deterministic simulation testing	Deployed	Turmoil-based (Tokio Project, 2026)
Per-peer resource limits (bandwidth, storage)	Deployed	Capability-relative default budget, operator-overridable
Sybil-resistant peer identity / location	Open	Defenses are application-layer (see Section 6)
Network-level incentive / abuse layer	Open	Application-layer reputation contracts are the intended mechanism

The single biggest gap between this paper’s narrative and the deployed system is congestion control. The platform supports pluggable congestion controllers but the deployed default is a token-bucket fixed-rate scheme. LEDBAT++ and BBR variants exist in the codebase as alternatives, but neither has yet been adapted to the workload satisfactorily. Real congestion control across heterogeneous peer-to-peer links remains open work.

7.3 Where the Platform Fits

Contract-defined merge, summary/delta synchronization, small-world routing, and delegates support applications whose conflicts can be resolved by application-defined merge. Examples include group chat, collaborative documents, social feeds, reputation systems, asynchronous inboxes, and identity directories. A working chat implementation (The Freenet Project, 2026b) and an anonymous identity scheme (The Freenet Project, 2026a) are deployed on the live network.

7.4 Where the Platform Does Not Fit

Several application classes remain poorly suited to the model.

Strictly linearizable transactions. A defense against double-spending fungible tokens requires that two attempts to spend the same balance cannot both succeed. The merge model cannot express “one of these updates is rejected unconditionally”; merge is total. Concretely, fungible-token transfer with platform-level double-spend protection cannot be expressed by a single contract in the current model. Applications that need a strong-consistency primitive must either contain the inconsistency inside a contract whose validity predicate detects and rejects the second update (which forces the conflict into application code, with the loser’s state restored on detection) or layer an auxiliary consensus mechanism on top of the platform.

The limitation extends beyond currency. A reputation system that only *accumulates* endorsements is monotone and fits the merge model. Revoking a credential, slashing a stake, or reducing a balance introduces the same requirement to reject a conflicting second update. Applications that otherwise fit the model may therefore require an additional consistency mechanism for non-monotone operations.

Abuse-resistant open posting. A purely-open contract is simple to write as a set under union and equally simple to flood. Practical contracts in this class restrict writes to signed records by approved keys, charge a proof-of-work fee per update, or require credentials issued by a reputation contract. The platform’s transport-layer rate limiting and per-neighbor resource accounting do provide a partial floor: a neighbor consuming excessive resources is disconnected, which caps the volume any single attacker can push through any single peer. That bound is useful but limited; it does not stop a distributed flood from many cheap identities, nor a slow-burn pattern that stays within per-peer limits. The platform has no application-level spam defense beyond that transport floor.

Coordinated DoS. Per-peer resource limits protect a peer against a single misbehaving neighbor, not against coordinated traffic from many neighbors using cheap identities. Defenses here probably require accounting that persists across identities, which is the open incentive layer noted above.

Durable storage of cold state. Contract state persists only while at least one peer near $\ell(\mathcal{C})$ retains it. Each peer holds contracts in a byte-budgeted hosted set whose size defaults to a fraction of host memory. When over budget, the peer evicts the contract with the fewest dependent subscribers, prioritizing local client subscriptions over downstream subscribers and breaking ties by least-recent genuine GET or PUT access. Subscriber count affects eviction order but does not pin a contract. If necessary, the peer evicts a subscribed contract and removes its subscription.

Eviction removes the persisted state, parameters, and code blob from disk. Cold contracts with no readers or subscribers are therefore the first candidates. The platform does not recruit a replacement when a host evicts a contract; recovery requires a new PUT from a party that still holds the state. A contract may consequently disappear from every host and become unrecoverable.

The platform provides no durability guarantee for cold state. Dormant collaborative documents, unpolled inboxes, and identity records for inactive users may remain cold for long periods. Maintaining a minimum replica count independently of demand—through active repair, incentivized pinning, or archival contracts—remains open work.

7.5 Open Problems

The following questions require further implementation or empirical evaluation.

1. **Scaling characterization.** The single 24-hour data point in Section 5.7 is consistent with logarithmic-diameter routing at the deployed network size. Characterization requires measurements across orders of magnitude in N . The current deployment cannot provide that range, and the deterministic simulation harness must be extended to emit hop-count-versus- N data before it can serve as an alternative.
2. **Failure modes of contract-defined merge.** A contract whose merge is not associative or idempotent does not converge. The runtime detects some observable non-idempotency by reapplying identical inputs and checking whether the result reaches a fixpoint. When detected, it suppresses commits and contract egress for a bounded interval; repeated detections lengthen that interval. This is containment, not verification: it does not establish associativity, commutativity, general idempotence, or sync soundness. Better diagnostics and broader platform-checked invariants remain open work. A near-term improvement would be a property-based test harness that generates states and updates to exercise associativity, commutativity, idempotence, and sync soundness (Property 1). Such a harness cannot constrain a malicious author, but it would provide honest authors with a repeatable CI workflow.
3. **Congestion control.** The fixed-rate default remains temporary while the alternative controllers are adapted and evaluated for this workload.
4. **Sybil-resistant identity.** Ghost Keys demonstrates an application-layer approach, but no general mechanism exists. Identity and reputation contracts are routed through the location layer they are intended to secure. An adversary with enough address prefixes to crowd a contract's location can serve poisoned reads and prevent honest writes from reaching its canonical replicas (Section 6). Prefix collisions also reduce the geographic diversity of honest peers. Resolving this circular dependency requires ring-distance separation among replicas of safety-critical contracts, demonstrable geographic diversity near their locations, or out-of-band trust anchors for the contracts that bootstrap the rest.
5. **Incentive / abuse layer.** The network needs a mechanism to resist coordinated denial of service without centralized admission control or excessive barriers to joining.
6. **Durability of cold state.** A contract that no one reads, subscribes to, or updates can be evicted from every host. A minimum replica count maintained through active repair, incentivized pinning, or archival contracts must avoid creating an unbounded free-storage abuse surface.

8 Related Work

Distributed hash tables. Chord (Stoica et al., 2001), Pastry (Rowstron and Druschel, 2001), and Kademlia (Maymounkov and Mazières, 2002) established the structured-overlay model inherited by Freenet's ring. They organize peers by hashed identifiers in a ring or XOR-metric space and provide $O(\log n)$ lookup through maintained routing tables. Freenet stores mutable, contract-defined state at each key and constructs its topology through the mechanism in Section 5, without explicit finger tables.

Content-addressed storage. The original Freenet (Clarke et al., 2001) introduced content-hash keys (CHKs) for immutable data around 1999–2000. IPFS (Benet, 2014) later applied content addressing to web deployment over its Bitswap block exchange. In both systems, external

naming or application layers provide mutability. Freenet contracts instead bind mutable state and its consistency rules to content-addressed code.

CRDTs. Conflict-free Replicated Data Types (Shapiro et al., 2011) provide the algebraic foundation for the contract layer. The contract interface implements the state-based (CvRDT) model while allowing applications to supply both the lattice and a summary/delta synchronization protocol. Antidote and AntidoteDB (Akkoorath et al., 2016) similarly provide CRDTs for geo-distributed databases, but expose a predefined CRDT vocabulary.

Bayou. Bayou (Terry et al., 1995) is the closest historical precedent for contracts. In 1995 it used application-supplied merge procedures on trusted replica servers to reconcile concurrent writes. Bayou procedures had database access and were configured per database. Freenet executes content-addressed merge code as sandboxed WebAssembly on untrusted hosts and restricts it to an idempotent commutative monoid. These restrictions allow replicas to converge without the platform inspecting the merge implementation or trusting its host.

Anti-entropy and delta sync. Dynamo (DeCandia et al., 2007) and its descendants use Merkle anti-entropy to reconcile replicas. Delta-state CRDTs (Almeida et al., 2018) formalize state-based synchronization that transfers only changes. Freenet follows the same general protocol shape, with contract-supplied lattices and delta encodings operating over a small-world routing layer instead of a fully connected replica set.

Local-first software and CRDT libraries. Local-first software (Kleppmann et al., 2019) favors convergence on user devices over server-mediated consistency. Automerge (Automerge contributors, 2026) and Y.js (Y.js contributors, 2026) provide composable CRDT structures such as JSON-like trees, sequences, and counters while leaving transport to the application. Those libraries guarantee convergence when applications remain within their primitives. Freenet lets each contract define its algebra and synchronization protocol, placing the corresponding correctness obligation on the contract author.

Secure Scuttlebutt. SSB (The SSB community, 2026) centers on per-user append-only signed logs replicated by gossip on demand. Such a feed can be represented as a Freenet contract whose merge is set union over signed records. Freenet generalizes the state structure and replication protocol beyond logs while targeting many of the same applications, including private messaging, blogging, and social feeds.

Hypercore. The Hypercore protocol family (Holepunch, 2026) provides content-addressed, append-only signed logs replicated peer-to-peer, with Hyperbee and Hyperdrive adding higher-level data structures. A Hyperbee resembles a contract with library-defined merge semantics, while a local key-management agent over Hypercore serves a role similar to a delegate.

Small-world routing. Kleinberg’s result (Kleinberg, 2000) provides the theoretical basis for the routing layer; the original Freenet (Clarke et al., 2001) predated and informally anticipated parts of it. Symphony (Manku et al., 2003) combines short-range ring links with $1/d$ long-range links sampled from a harmonic distribution. Freenet likewise protects its closest successor and predecessor, but continuously adjusts the long-range distribution: a joiner targets an under-represented region of its connections, and the receiver favors candidates that close large gaps. To our knowledge, this combination has not been analyzed in the structured-overlay literature.

Performance-aware routing. Adaptive routing in peer-to-peer overlays has a long history. Freenet uses isotonic regression, a standard calibration method (de Leeuw et al., 2009), as a monotone distance-based baseline, then combines it in a weighted ensemble with Renegade’s k-nearest-neighbor predictions (Clarke, 2026) to represent peer–contract interactions. The two components predict the same outcomes; Renegade is not trained as a residual correction to the isotonic model.

Blockchain platforms. Ethereum (Wood, 2014) and successor smart-contract platforms execute a globally linearized replicated state machine. Freenet instead provides per-contract eventual consistency with application-defined merge. It is intended for interactive applications with mergeable updates and does not support applications that require a single global transaction order.

Federated systems. Matrix (The Matrix.org Foundation, 2026) is a server-based federated system with a similar application surface. Its state-resolution algorithm reconciles divergent room states by a deterministic application-defined rule. Freenet applies this approach in a peer-to-peer setting and permits arbitrary idempotent commutative monoids.

The original Freenet. The original Freenet (Clarke et al., 2001) provided anonymous, content-addressed storage for immutable files. Its heuristic forwarded requests to peers whose previous successful routes most resembled the requested key, anticipating aspects of small-world greedy routing without assigning peers explicit locations. The present system retains content-addressed naming and the small-world routing intuition. It adds explicit locations, contract-defined mutable state, delegates, and user interfaces, while moving anonymity to application-layer protocols.

9 Conclusion

Applications on Freenet define their state algebra in sandboxed WebAssembly. A contract-supplied summary/delta protocol synchronizes that state across a mesh of neighboring co-hosts, while demand-gated subscription leases keep hosting aligned with active use. The underlying small-world ring protects a nearest successor and predecessor while shaping its remaining links through gap-based **CONNECT** targeting and Kleinberg-style acceptance scoring. Forwarding uses an isotonic distance baseline blended with a k-nearest-neighbor predictor. Delegates isolate private state and sensitive operations behind a platform-enforced trust boundary.

This architecture targets decentralized group chat, collaborative documents, social feeds, reputation systems, and other applications with mergeable state. Several mechanisms remain experimental or unimplemented. In particular, evaluating asymptotic scaling will require measurements across network sizes rather than further design analysis alone.

Acknowledgements

Special thanks to Ignacio Duarte Gomez and Hector Alberto Santos Rodriguez for their substantial engineering contributions to Freenet, and to Steven Starr for his invaluable support with outreach, fundraising, and strategic guidance.

References

- Deepthi Devaki Akkoorath, Alejandro Z. Tomsic, Manuel Bravo, Zhongmiao Li, Tyler Crain, Annette Bieniusa, Nuno Preguiça, and Marc Shapiro. Cure: Strong semantics meets high availability and low latency. In *Proceedings of the 36th IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 405–414, 2016.
- Paulo Sérgio Almeida, Ali Shoker, and Carlos Baquero. Delta state replicated data types. *Journal of Parallel and Distributed Computing*, 111:162–173, 2018.
- Automerge contributors. Automerge: A CRDT library for building local-first applications, 2026. <https://automerge.org/>, accessed 2026-05-24.
- Richard E. Barlow, D. J. Bartholomew, J. M. Bremner, and H. D. Brunk. *Statistical Inference Under Order Restrictions: The Theory and Application of Isotonic Regression*. Wiley, 1972.
- Juan Benet. IPFS — content addressed, versioned, P2P file system, 2014. arXiv:1407.3561.
- Christopher Berner. redb: An embedded key-value database in pure Rust, 2026. <https://github.com/cberner/redb>, accessed 2026-05-24.
- Bytecode Alliance. Wasmtime: A standalone runtime for WebAssembly, 2026. <https://wasmtime.dev/>, accessed 2026-05-24.
- Ian Clarke. Renegade: Zero-config nonparametric supervised learning for Rust, 2026. <https://github.com/sanity/reneade>, accessed 2026-08-09.
- Ian Clarke, Oskar Sandberg, Brandon Wiley, and Theodore W. Hong. Freenet: A distributed anonymous information storage and retrieval system. In *Designing Privacy Enhancing Technologies: International Workshop on Design Issues in Anonymity and Unobservability*, pages 46–66. Springer, 2001.
- Jan de Leeuw, Kurt Hornik, and Patrick Mair. Isotone optimization in R: Pool-adjacent-violators algorithm (PAVA) and active set methods. *Journal of Statistical Software*, 32(5):1–24, 2009.
- Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voshall, and Werner Vogels. Dynamo: Amazon’s highly available key-value store. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pages 205–220, 2007.
- Bryan Ford, Pyda Srisuresh, and Dan Kegel. Peer-to-peer communication across network address translators. In *USENIX Annual Technical Conference*, pages 179–192, 2005.
- Holepunch. Hypercore Protocol: A streaming, append-only, signed log for peer-to-peer applications, 2026. <https://holepunch.to/>, accessed 2026-05-24.
- Jon Kleinberg. The small-world phenomenon: An algorithmic perspective. In *Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing (STOC)*, pages 163–170, 2000.
- Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, pages 154–178. ACM, 2019.

- Gurmeet Singh Manku, Mayank Bawa, and Prabhakar Raghavan. Symphony: Distributed hashing in a small world. In *Proceedings of the 4th USENIX Symposium on Internet Technologies and Systems (USITS)*, pages 127–140, 2003.
- Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the XOR metric. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, pages 53–65. Springer, 2002.
- Antony Rowstron and Peter Druschel. Pastry: Scalable, decentralized object location, and routing for large-scale peer-to-peer systems. In *Middleware 2001*, pages 329–350. Springer, 2001.
- Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 386–400. Springer, 2011.
- Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, and Hari Balakrishnan. Chord: A scalable peer-to-peer lookup service for internet applications. In *Proceedings of ACM SIGCOMM*, pages 149–160, 2001.
- Douglas B. Terry, Marvin M. Theimer, Karin Petersen, Alan J. Demers, Mike J. Spreitzer, and Carl H. Hauser. Managing update conflicts in Bayou, a weakly connected replicated storage system. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP)*, pages 172–182, 1995.
- The Freenet Project. Ghost Keys: Anonymous, verifiable identities on Freenet, 2026a. <https://freenet.org/ghostkey/>, accessed 2026-05-24.
- The Freenet Project. River: A decentralized group chat on Freenet, 2026b. <https://github.com/freenet/river>, accessed 2026-05-24.
- The Matrix.org Foundation. The Matrix specification, 2026. <https://spec.matrix.org/>, accessed 2026-05-24.
- The SSB community. Secure Scuttlebutt: A protocol for decentralized social applications, 2026. <https://scuttlebutt.nz/>, accessed 2026-05-24.
- Tokio Project. Turmoil: A framework for testing distributed systems, 2026. <https://docs.rs/turmoil>, accessed 2026-05-24.
- Andrew Tridgell. *Efficient Algorithms for Sorting and Synchronization*. PhD thesis, The Australian National University, 1999.
- Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger, 2014. Ethereum Project Yellow Paper.
- Y.js contributors. Y.js: Shared data types for building collaborative software, 2026. <https://yjs.dev/>, accessed 2026-05-24.